

6.1 Turing Machine as a Physical Computing Device

Turing machines, first described by Alan Turing in (Turing 1937), are simple abstract computational devices intended to help investigate the extent and limitations of what can be computed.

Neither DFA nor PDA can be viewed as a truly general models for computers, since they are not capable of recognizing even such simple language $L=\{a^n b^n c^n \mid n \geq 0\}$. we can visualize Turing machine as a physical computing device that can be represented as a diagram below.

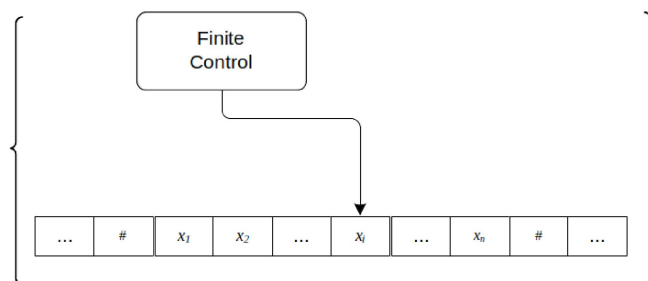


Fig: Turing Machine

The machine consists of a finite control which can be in any of a finite set of states. There is a tape divided into squares or cell; each cell can hold any one of a finite number of symbols. Initially, the inputs which is a finite-length string of symbols chosen from the input alphabet, is placed on the tape. All other tape cells, extending infinitely to the left and right, initially hold a special symbol called the blank. The blank is a tape symbol but not an input symbol, and there may be other tape symbols besides the input symbols and the blank, as well. There is a tape head that is always positioned at one of the tape cells. The Turing machine is said to be scanning that cell. Initially, the tape head is at the leftmost cell that holds the input.

A move of the Turing machine is a function of the state of the finite control and the tape symbol scanned. In one move, the Turing machine will:

1. Change state. The next state optionally may be the same as the current state.

2. Write a tape symbol in the cell scanned. This tape symbol replaces whatever symbol was in that cell. Optionally, the symbol written may be the same as the symbol currently there.
3. Move the tape head left or right. In our formalism we require a move, and do not allow the head to remain stationary. This restriction does not constrain what a Turing machine can compute, since any sequence of moves with a stationary head could be condensed, along with the next tape-head move, into a single state change, a new tape symbol, and a move left or right.

In Summary:

- Unrestricted memory: an infinite tape
 - A finite state machine that reads/writes symbols on the tape
 - Can read/write anywhere on the tape
 - Tape is infinite in one direction only (other variants possible)
- Initially, tape has input and the machine is reading (i.e., tape head is on) the leftmost input symbol.
- Transition (based on current state and symbol under head):
 - Change control state
 - Overwrite a new symbol on the tape cell under the head
 - Move the head left, or right.

6.2 Turing Machine VS Finite Automata

1. A TM can both write on the tape and read from it;
A FA can only read its input
2. The read/write head of a TM can move both to the left and to the right;
A FA can move in one direction only.
3. The tape of a TM is infinite;
The input of a FA is finite.
4. The special states of a TM for rejecting and accepting the input take immediate effect;
FA terminates when control reads entire input tape.

6.3 Formal Definition of Turing Machine

We describe a TM by the 7-tuple, $M = (Q, \Sigma, \tau, \delta, q_0, \#, F)$ where,

Q : The finite set of states of the finite control.

Σ : The finite set of input symbols.

Γ : The finite set of tape alphabets.

δ : The transition function. The arguments of $\delta(q, X)$ are a state q and a tape symbol X . The value of $\delta(q, X)$, if it is defined, is a triple (p, Y, D) , and

→ p is the next state, in Q .

→ Y is the symbol, in τ written in the cell being scanned, replacing whatever symbol was there.

→ D is a direction, either L or R , standing for "left" or "right" respectively. The direction in which the head moves.

i.e.

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$$

q_0 : The start state, a member of Q , in which the finite control is found initially.

$\#$: The blank symbol. This symbol is in τ but not in Σ ; i.e., it is not an input symbol. The blank appears initially in all but the finite number of initial cells that hold input symbols.

F : The set of final or accepting states, a subset of Q .

6.4 Instantaneous description (ID) or Configuration

An instantaneous description or configuration of a Turing Machine requires:

1. The state the Turing machine is in.
2. The contents of the tape
3. The position of the tape head on the tape.

This is written in the form:

$$x_i \dots x_j q_m x_k \dots x_l$$

Where x 's are the symbol on the tape, q_m is the current state and the tape head is on the square containing x_k (The symbol immediately following q_m). The move of a Turing Machine can therefore be expressed as a pair of instantaneous descriptions, separated by “ $\vdash$ ”.

For example, $\delta(q_5, b) = (q_8, c, R)$

Then possible move can be: $abbabbq_5babb \vdash abbabcq_8abb$

As a Turing Machine computes, changes occur in the current state, the current tape contents, and the current head location. A setting of these three items is called a *configuration* of the Turing Machine. For example, $1011q_701111$ represents the *configuration* when the tape is 101101111 , the current state is q_7 , and the head is currently on the second 0. The following figure represents a Turing Machine with the configuration.

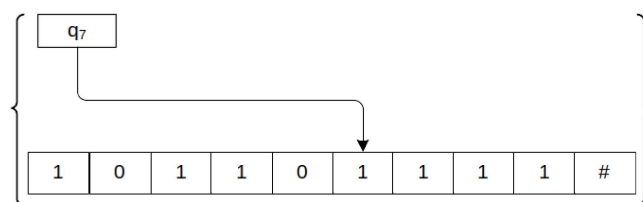


Fig. A Turing machine with configuration $1011q_701111$.

6.5 Language Accepted by TM

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \#, F)$ be a Turing Machine (TM). Then the language accepted by the TM is defined by:

$$L(M) = \{\omega \in \Sigma^+ : q_0\omega \vdash x_1 q_f x_2 \text{ for some } q_f \in F, x_1, x_2 \in \Gamma^*\}.$$

6.6 Designing Turing Machine

1. Design a Turing Machine that accepts the language $L = \{ a^n b^n : n \geq 1 \}$.

Solution:

we solve the problem in the following fashion. Starting at the leftmost a , we check it off by replacing it with some symbol, say x . We then let the read-write head travel right to find the leftmost b , which in turn is checked off by replacing it with another symbol, say y . After that, we go left again to the leftmost a , replace it with an x , then move to the leftmost b and replace it with y and so on. Traveling back and forth this way, we match each a with a corresponding b . If after some time not any a 's or b 's remain, then the string must be in L .

Working out the details, we arrive at a complete solution for which

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$F = \{q_4\},$$

$$\Sigma = \{a, b\},$$

$$\Gamma = \{a, b, x, y, \#\}.$$

The transition can be broken into several parts. The set:

$$\delta(q_0, a) = (q_1, x, R),$$

$$\delta(q_1, a) = (q_1, a, R),$$

$$\delta(q_1, y) = (q_1, y, R),$$

$$\delta(q_1, b) = (q_2, y, L).$$

replaces the leftmost a with an x , then causes the read-write head to travel right to the first b , replacing it with a y . When the y is written, the machine enters a state q_2 indicating that a symbol a has been successfully paired with a symbol b .

The next set of transitions reverses the direction until an x is encountered, reposition the read-write head over the leftmost a , and returns control to the initial state.

$$\delta(q_2, y) = (q_2, y, L),$$

$$\delta(q_2, a) = (q_2, a, L),$$

$$\delta(q_2, x) = (q_0, x, R).$$

We are now back at the initial state q_0 , ready to deal with the next input a and b . After one pass through this part of the computation, the machine will have carried out the partial computation.

$$q_0 a a \dots a b b \dots b \vdash^* x q_0 a \dots a y b \dots b$$

After two passes we will have completed the partial computation:

$$q_0 a a \dots a b b \dots b \vdash^* x x q_0 \dots a y y \dots b$$

and so on, indicating that the matching process is being carried out properly.

When the input is a string $a^n b^n$, the rewriting continues this way, stopping only when there are no more a 's to be erased. When looking for the leftmost a , the read-write head travels left with the machine in state q_2 . When an x is encountered, the direction is reversed to get the a . But now, instead of finding an a it will find a y . To terminate, a final check is made to see if all a 's and b 's have been replaced (to detect input where an a follows a b). This can be done by:

$$\delta(q_0, y) = (q_3, y, R)$$

$$\delta(q_3, y) = (q_3, y, R)$$

$$\delta(q_3, \#) = (q_4, \#, R)$$

The particular input $aabb$ gives the following successive instantaneous descriptions.

$$\begin{array}{l} q_0 a a b b \vdash x q_1 a b b \vdash x a q_1 b b \vdash x q_2 a y b \vdash x q_0 a y b \vdash x x q_1 y b \vdash x x y q_1 b \vdash x x q_2 y y \vdash \\ x q_2 x y y \vdash x x q_0 y y \vdash x x y q_3 y \vdash x x y q_3 y \vdash x x y y q_3 \# \vdash x x y y \# q_4 \# \end{array}$$

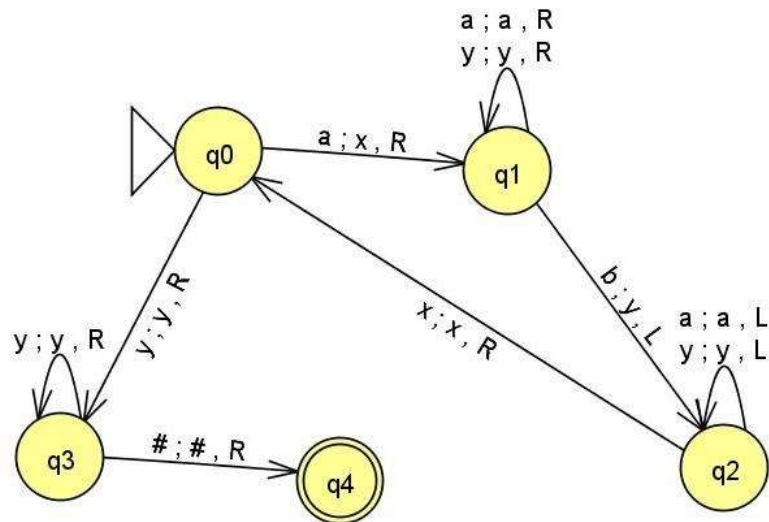


Fig: Transition diagram for a TM that accepts strings of the form $a^n b^n$.

2. Design a Turing Machine that accepts the language $L = \{ a^n b^n c^n : n \geq 1 \}$.

The idea behind this machine i.e. TM is also similar to previous problem. We replace each a , b and c by x , y and z respectively during right and left movement of head. So, this is the extension of the language $\{a^n b^n : n \geq 1\}$ and the designed TM will be very much similar to the above example.

6.7 Turing Machine as Transducers

Computation means to transform the input into its output. We can use TM for the same purpose and acts as a transducer. The input is the non-blank symbols on the tape and the output after computation is left on the tape.

The TM implementation in terms of function f is defined by:

$$f(x) = \hat{x} \text{ provided that } q_0x \vdash_M^* q_f f(x).$$

A function f with domain D is said to be **Turing Computable** or **Computable** if there exists some TM $M = (Q, \Sigma, \Gamma, \delta, q_0, \#, F)$ such that

$$q_0x \vdash_M^* q_f f(x), q_f \in F \text{ for all } x \in D.$$

Example:

Design a Turing Machine (TM) that add two given integers.

Let x and y are two given integers to be added and the input for the TM is $0^x \# 0^y$. If we design TM that can able to remove the $\#$ from the input which brings all 0's together and is the output for $x+y$. The process is:

- (a) The separating $\#$ is replaced by a 0.
- (b) The right most 0 is replaced by a $\#$.

So, the TM for the purpose is:

$$M = (\{q_0, q_1, q_2, q_3, q_4\}, \{0\}, \{0, \#\}, \delta, q_0, \#, \{q_4\}).$$

The transitions are:

$$\begin{aligned} \delta = \{ & \\ & \delta(q_0, 0) = (q_0, 0, R), \\ & \delta(q_1, 0) = (q_1, 0, R), \\ & \delta(q_2, 0) = (q_3, \#, L), \\ & \delta(q_3, 0) = (q_3, 0, L), \\ & \delta(q_0, \#) = (q_1, 0, R), \\ & \delta(q_1, \#) = (q_2, \#, L), \\ & \delta(q_3, \#) = (q_4, \#, R) \\ & \}. \end{aligned}$$

Here, M starts from ID $q_0 0^x \# 0^y$, moves right until seeking the blank $\#$. M changes state to q_1 and the separated $\#$ is replaced by 0. It again starts to move towards right. On reaching the right end, it changes to state q_2 and replaces the rightmost 0 by $\#$ which results the current state to q_3 . Then it moves to the left end which is the beginning of the input string. Finally, It halts at the final state q_4 .

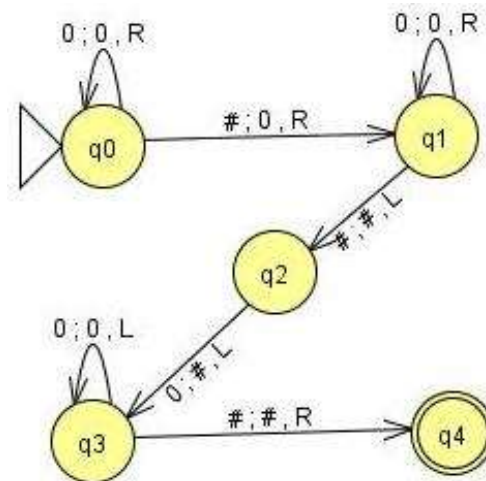


Fig: Transition diagram for a TM that computes $x+y$.

6.8 Turing's Thesis (Church's Turing Hypothesis)

Alan Turing proposed the hypothesis called Turing's Thesis which states that any computation that can be done or carried out by *mechanical means* can be performed by some Turing Machine. So, What is *mechanical means* ?

Mathematical computation for the same problems are worked by:

1. Alonzo Church → Lambda Calculus
2. Emil Post → Production System
3. Noam Chomsky → Unrestricted Grammars
4. Stephen Kleene → Recursive Function Theory
5. Raymond Smullyn → Formal Systems

To define the term *mechanical means*, it would require some other abstract model. So, if we view the Turing thesis as mechanical computation then a *computation* is *mechanical* if and only if it can be performed by some Turing Machine. This is a formalize notion of an "effective procedure" which is called as *algorithm* these days.

All the formalism's in different mathematical fields i.e. mechanical computation led to:

1. **Turing's Thesis (Weak Form):** Anything that can be computed on any general purpose digital computer can also be computed by a Turing Machine.
2. **Turing's Thesis (Strong Form):** A Turing Machine can compute anything that can be computed.

No problem exists that is solvable by an algorithm for which TM can be constructed and also the alternative model for *mechanical computation* is not more powerful than the Turing Machine model. The *strong form* of Turing's Thesis states a relationship between mathematical concepts and the real world and cannot be proven yet.

Algorithm: An algorithm for a function $f : D \rightarrow R$ is a Turing Machine M , which given as input any $x \in D$ on its tape, eventually

halts with the correct answer $f(x) \in R$ on its tape. Specifically, we can require that

$$q_0x \vdash_M^* q_f f(x), q_f \in F \text{ for all } x \in D.$$

6.9 Variations of Turing Machines

Standard Turing Machine: At any transition made on TM, the tape head either moves left or right. If we add 'stay' option to it i.e. don't move to the set $\{L, R\}$ then the resultant TM is called TM with stay option and are equivalent to *Standard Turing Machine*. So, a TM with stay option replaces the transition as follows:

$$\delta : Q \times \Gamma = Q \times \Gamma \times \{L, R, S\}.$$

Turing Machine with Semi-Infinite Tape: has non-blank input at the extreme left of the tape.

Offline Turing Machine: has two tapes, one tape being read only for input and another being read-write tape that is initially blank.

Multi-tape Turing Machine: has finite number of tapes, each having its own independent control head.

Binary Turing Machine: has exactly two symbols as tape alphabets (Σ).

N-track Turing Machine: A TM in which each square of the tape holds n -tuple of symbols from the tape alphabet is said to be an N -Track TM.

Non Deterministic Turing Machine:

We can also imagine that a Turing Machine can act non-deterministically similar to the case of Finite Automata and Push Down Automata. In standard Turing Machine to each pair of current state (except the halt state) and the symbol being scanned, there is a unique triplet comprising the next state, unique action in terms of writing a symbol in the cell being scanned and the motion if any, to the right or left. However in case of Non-Deterministic Turing Machine (NDTM) , to each pair (q, a) with q as current state and a as symbol being scanned, there may be a finite set of triplets $\{(q_i, a_i, m_i); i = 1, 2 \dots\}$ of possible moves. The set of triplets may be empty that is for some particular (q, a) the TM may not have the next move.

A non-deterministic TM is a 7-tuple.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \#, F)$$

where each component has the same meaning as of the standard Turing machine except the transition function, δ :

$$\begin{aligned} \delta: (Q \times \Gamma) &\rightarrow \text{power set of } (Q \times \Gamma \times \{L, R, S\}). \text{ i.e.} \\ \delta: (Q \times \Gamma) &\rightarrow 2^{(Q \times \Gamma \times \{L, R, S\})} \end{aligned}$$